techniques, so you could roll your own fluent interface:

```
task 'build', 'run the build', ->
  clean('compiled')
  .then(compile 'app')
  .then(copy 'content', 'compiled')
  .then(createArtifact 'compiled', VERSION)
  .then(-> console.log 'done')
  .run()
```

Or perhaps you could implement your own function to handle dependencies:

```
task 'build', 'run the build', depends ['clean', 'compile',
'copy'], ->
```

Or you could implement your own syntax by extending the CoffeeScript compiler (see chapter 8). Many things are possible, but for now, it's time to recap.

12.6. Summary

In this chapter you learned how to structure a simple application into discrete modules on the server and on the client and then how to build, test, and release it.

That marks the end of your discovery of the current world of CoffeeScript, but it doesn't mark the end of your journey. Your journey continues into the future, where in the next chapter you'll look at where JavaScript is headed and at how CoffeeScript fits into that picture.